

Termination Checking

An Annotated Bibliography

Ivan Wei (ivanwei@umich.edu)
University of Michigan

1 Overview

General recursion is not compatible with dependently typed programming languages. An infinitely recursive term can have any type, including types that are supposed to be empty like $\perp$ corresponding to false propositions. Also, since the subsumption rule requires type equivalence checks, which requires expression equivalence checks, for type-checking to be decidable, expression equivalence must be decidable, which can't occur with non-terminating recursion.

Recursion is still useful to express algorithms in a proof assistant or for the sake of induction. Hence, efforts have been made to introduce limited forms of recursion with guaranteed termination.

One method is to introduce primitive recursor forms that work on certain types like the natural numbers and lists. These forms take a value to produce in the base case, and a function taking the result of recursive application on any predecessors in other cases. For example, consider the recursor on natural numbers given by the typing and evaluation rules

$$\begin{aligned}\Gamma \vdash \text{natind} : & \quad (P : \mathbb{N} \rightarrow \text{Type}) \\ & \rightarrow P(Z) \\ & \rightarrow ((x : \mathbb{N}) \rightarrow P(x) \rightarrow P(S(x))) \\ & \rightarrow (n : \mathbb{N}) \rightarrow P(n)\end{aligned}$$

$$\begin{aligned}\text{natind } P \ z \ s \ Z & \equiv z \\ \text{natind } P \ z \ s \ S(n) & \equiv s \ n \ (\text{natind } P \ z \ s \ n)\end{aligned}$$

The main effort sought to answer two questions: how do we allow for the expression of a greater class of terminating general recursive functions

in dependent type theory, and how do we make it easier for programmers to express such functions in dependent type theory?

The initial direction pursued in the field was to improve the recursor to work with arbitrary inductive types. Nordström [6] gave rules for a recursor that works, for any given type, on the *accessible* values of that type. Although it is sometimes fairly difficult to provide proofs of accessibility and in general use this recursor, it is exceptionally flexible in the kinds of algorithms it can express due to the freedom to define the relevant binary relation on the type. Paulin-Mohring [7] instead provides some explicit syntactic forms to create custom inductive definitions of recursors in the Calculus of Constructions. The ultimate problem with these approaches is that the recursors are even more unwieldy to use than those derived from certain specific types. Nordström’s approach needs lots of boilerplate to define accessibility and prove it for values provided to his recursor, while Paulin-Mohring’s approach comes with some auxiliary functions that can make the inductive definitions more complex.

Programmers are used to working with recursion, rather than these complex recursors. This observation drove research in the other direction: sacrificing some expressability in favor of ease of use. The key observation is that many recursive functions can be written as a containing a case analysis into constructors followed by recursive call on the constructor’s components. These components are called *structurally smaller* than the scrutinee, and it can be shown that structural recursive functions that only make recursive calls on structurally smaller arguments will always terminate, as shown in Abel and Altenkirch’s work [1]. Hence, structural recursive functions can be expressed directly in dependent type theory. This is advantageous, because they are easier to understand for programmers. The problem is that several recursive algorithms cannot be expressed using a single structural recursive function on a single argument. And so, with the ease of use problem solved, research turned towards widening the class of recursive algorithms structural recursion could express. Abel and Altenkirch’s work allows for mutual recursion, multiple recursive calls, and multiple arguments with structural recursion. Bove and Capretta [5], taking inspiration from Nordström’s idea of accessibility, translate a large class of general recursive algorithms to structural recursive ones on proofs of accessibility/termination on the input.

Barthe, et. al [3] introduce sized types because of the lack of expressivity of structural recursion especially before Bove and Capretta’s work. They also describe implementation difficulty of checking the structural ordering condition. Sized types involve the annotation of types with sizes

which bound the depth of a constructor tree. They describe how certain forms increase and decrease that size, like fixpoints, constructors, and case analysis. Namely, the arguments to recursive calls inside a fixpoint must always be of lesser size, giving termination. This was easier to implement and could express more kinds of functions. However, since these annotations now permeate through types, more work had to be done to incorporate sizes into different type theories, including those with dependent types and polymorphism. For example, Blanqui [4] extended this system of size annotation to the Calculus of Algebraic Constructions.

The current state of open research seems to be centered around sized types. One direction of research is into size inference and polymorphism [2], so that not all type annotations have to also be provided with size annotations. Another direction is the relationship between universe checking and size checking, since they seem to have some similarities and could share techniques. In addition, the only proof assistant that has adopted sized types is Agda, so there is more work to be done on their adoption to other major proof assistants.

2 Nordström’s recursor

Nordström [6] observed that recursors in common type theories were defined only on natural numbers and inductive sets (i.e., the set of lists). To solve this, he sought to generalize the recursor to other types.

Definition 1. Let A be a set equipped with the binary relation $\prec$. $a \in A$ is said to be **accessible** provided that all descending chains $a \succ a_0 \succ \dots$ terminate. The set of all accessible elements of A is denoted $\text{Acc}(A, \prec)$.

Definition 2. A is said to be **well-founded** provided that all its elements are accessible.

Remark 1. If b is accessible and $a \prec b$, then a is also accessible. Any infinite descending chain starting with a would induce an infinite descending chain starting with b .

Remark 2. Types can be viewed as sets that terms belong to.

In this context, Nordström introduces the operator rec , which can be applied to values $p : \text{Acc}(A, \prec)$ for all types A and binary relations $\prec$ on A and functions e which are applied to values of type A and functions of its own type. He gives the following rule for rec :

$$\frac{p \in \text{Acc}(A, \prec) \quad e(x, y) \in C(x) \quad \left(\begin{array}{l} \forall x \in \text{Acc}(A, \prec) \\ \forall y \text{ s.t. } \forall z \in A \text{ s.t.} \\ z \prec x, y(z) \in C(z) \end{array} \right)}{\text{rec}(e, p) \equiv e(p, \text{rec}(e)) \in C(p)} \quad \text{Acc-} \equiv$$

If the function e is well-typed (and thus terminates) on all accessible values x and functions y that are well-typed on all inputs $z \prec x$, then $\text{rec}(e, p)$ is well-typed. The expectation is that y is the recursive call $\text{rec}(e)$, and thus y only guarantees termination if provided with a smaller (accessible) value under $\prec$ than x . If you prove that e terminates under this assumption on x (akin to a proof of an inductive step), then the whole expression $\text{rec}(e, p)$ will terminate. The descending chain given by whatever smaller arguments e passes to y in each iteration will always terminate because p and x are assumed accessible.

We can define an alternate formulation of the rec operator which makes more types clear. First, define

$$\Gamma \vdash \text{acc_helper} : (b : \text{Acc}(A, \prec)) \rightarrow (a : A) \rightarrow (a \prec b) \rightarrow \text{Acc}(A, \prec)$$

This helper converts elements of A that are accessible (by being less than another accessible value) into an element of $\text{Acc}(A, \prec)$. Then the rec operator can be expressed by the rules

$$\begin{array}{l} \Gamma \vdash \text{rec} : (C : A \rightarrow \text{Type}) \\ \quad \rightarrow ((x : \text{Acc}(A, \prec)) \rightarrow ((z : A) \rightarrow (z \prec x) \rightarrow C(z)) \rightarrow C(x)) \\ \quad \rightarrow (p : \text{Acc}(A, \prec)) \\ \quad \rightarrow C(p) \end{array}$$

$$\text{rec } C \ e \ p \equiv e \ p \ (\text{fun } q \rightarrow \text{fun } pf \rightarrow \text{rec } C \ e \ (\text{acc_helper } p \ pf \ q))$$

The second argument to rec is indeed the inductive step analogous to e in the original rule, also taking x and y as analogous arguments.

Nordström evaluates his solution by making the bold claim that all iterating constructs in type theory can be expressed as a combination of this rec operator and pattern matching. He doesn't provide a formal statement or proof, but instead provides an example by defining natrec using rec and natcases .

The lack of a formal statement or serious evaluation is one of the limitations of the paper. Another limitation is that the initial p passed to rec

needs to be proven to be an element of $\text{Acc}(A, \prec)$. For well-founded types like $\mathbb{N}$ and lists, all values are accessible, so this fact is easy to prove. But proving it in a general case for different A or different $\prec$ may be nonobvious. Finally, the use of a recursor is unfamiliar to programmers who are used to writing recursive algorithms.

3 Paulin-Mohring’s schema for inductive definitions

Paulin-Mohring [7] also motivates her paper with the desire to generalize the recursor through *inductive definitions*. By the time the paper was written, inductive definitions had already been implemented in Coq, but needed formal treatment. She does so in this paper.

Definition 3. An **arity of sort** s is given by terms in the form $Ar ::= s \mid (x : M) \rightarrow Ar$.

Definition 4. If A is an arity and X is of type A , then terms that are **strictly positive** with respect to X are given by $P ::= X \mid P \ m \mid (x : M) \rightarrow P$ where X does not occur in m or M .

Definition 5. If A is an arity, X is of type A , then **constructor types** are given by $C ::= X \mid C \ m \mid P \rightarrow C \mid (x : M) \rightarrow C$ where P is strictly positive with respect to X and X does not occur in m or M .

The introduction form of an inductive definition is given by the rule

$$\frac{\Gamma, X : A \vdash C_i : s}{\Gamma \vdash \text{Ind}(X : A)\{C_1 \mid \cdots \mid C_n\} : A}$$

where A is an arity of sort s and the C_i ’s are constructor types.

Let I be an inductive definition $\text{Ind}(X : A)\{C_1 \mid \cdots \mid C_n\}$. Constructor terms are given by the rule

$$\frac{\Gamma \vdash I : A}{\Gamma \vdash \text{Constr}(i, I) : [I/X]C_i}$$

For a constructor type C , $C\{X, Q, c\}$ corresponds to “constructor substitution” via substitution of Q for X , except that X is an arbitrary term and not a variable. Also, c is applied to any dependent arrow arguments.

Let I be an inductive definition $\text{Ind}(X : A)\{C_1 | \dots | C_n\}$ and s' be a sort. The elimination form of an inductive definition is then given by the rule

$$\frac{\begin{array}{c} \Gamma \vdash c : (I \ a_1 \ \dots \ a_m) \\ \Gamma \vdash Q : (x_1 : A_1) \rightarrow \dots \rightarrow (x_m : A_m) \rightarrow (I \ x_1 \ \dots \ x_m) \rightarrow s' \\ \Gamma \vdash f_i : C_i\{I, Q, \text{Constr}(i, I)\} \end{array}}{\Gamma \vdash \text{Elim}(c, Q)\{f_1 | \dots | f_n\} : Q \ a_1 \ \dots \ a_m \ c}$$

For a constructor type C , $C[X, F, f]$ represents the application of f on the components of the constructor and the result of a “recursive” call from F .

Then the reduction step is ι -reduction, which is given by the rule (for I as above)

$$\begin{array}{c} \text{Elim}(\text{Constr}(i, I) \ m, Q)\{f_1 | \dots | f_n\} \\ \longrightarrow_{\iota} \\ C_i[I, \text{fun}(x_1 : A_1) \rightarrow \dots \rightarrow \text{fun}(x_m : A_m) \rightarrow \\ \text{fun}(c : I \ x_1 \ \dots \ x_m) \rightarrow \text{Elim}(c, Q)\{f_1 | \dots | f_n\}, f_i] \ m \end{array}$$

This corresponds to one self-substitution step akin to a fixpoint operator.

Paulin-Mohring provides a translation of this calculus with inductive definitions to a different calculus CC_{μ} which instead extends dependent type theory with fixpoints. The surprising result is that ι -reduction above corresponds to a β -reduction in CC_{μ} , rather than a fixpoint reduction! Paulin-Mohring then leverages the strong normalization of CC_{μ} with respect to β -reduction to immediately show that her calculus is strongly normalizing.

The paper admits the limitation that the strict positivity condition is strong. Relaxing it could allow for mutual inductive definitions, though doing so might be less natural than directly generalizing the inductive definitions. Furthermore, one can broadly observe that using these inductive definitions is even more unfamiliar to programmers who are used to writing recursive functions, as the typing and reduction rules are complex, even to the point of requiring auxiliary definitions like $C\{X, Q, c\}$ and $C[X, F, f]$.

4 Abel and Altenkirch’s foetus calculus for structural recursion with call graphs

Abel and Altenkirch [1] motivate their approach to termination checking through comparison with standard functional programming languages. Recursors tend to be more difficult to understand, and using them can require

additional program constructs to express what can be easily written with general recursive calls. Instead, it can be observed that many recursive functions on inductive types are structured as an immediate pattern match on the argument followed by a recursive call using the branch components. For example, the paper presents a function that halves a natural number (taking the floor) as the following:

```
fun half 0 = 0
  | half (S 0) = 0
  | half (S (S n)) = S (half n)
```

With such functions, termination is proven intuitively by continued elimination on the argument eventually unravelling the term into one typed as the base case branch in the sum in the inductive type. More formally, to prove termination, all recursive calls must be performed on syntactically “smaller” elements. Intuitively, structural ordering is given by a component of a constructor being less than the constructed value.

Definition 6. Structural ordering is given by the following rules: for $R \in \{<, \leq\}$,

$$\frac{}{\Delta \vdash x \leq x} \leq \text{refl}$$

Case branch components are given as structurally smaller than the scrutinee in the context:

$$\frac{\Delta, y_i \leq s \vdash t_i R x (\forall i)}{\Delta \vdash \text{case}(s, y.\vec{t}) R x} R_{\text{case}}$$

Structural relations are preserved through projection:

$$\frac{\Delta \vdash t R x}{\Delta \vdash \pi_j(t) R x} R_{\text{pi}}$$

Structural relations are preserved through application:

$$\frac{\Delta \vdash f R x}{\Delta \vdash f a R x} R_{\text{app}}$$

Unfolding gives a strictly structurally smaller term:

$$\frac{\Delta \vdash t \leq x}{\Delta \vdash \text{unfold}(t) R x} R_{\text{unf}}$$

Now the termination checker checks that the arguments to the recursive calls in a function are strictly structurally smaller than the arguments to the

current call. Suppose all types are well-founded under structural ordering. Suppose a function f only ever calls itself on structurally smaller arguments. Consider the sequence of calls with arguments v_i

$$f\ v_0, f\ v_1, f\ v_2 \dots$$

where

$$v_0 > v_1 > v_2 > \dots$$

Since no infinite descending chains of elements exist in well-founded sets, this sequence of calls must terminate.

Thus, to show that all structural recursive functions terminate, it is sufficient to show all types in the calculus are well-founded. Abel and Altenkirch define a calculus called **foetus**, and prove exactly this. They provide a semantic set interpretation for values. They define the structural ordering in the codomain of this interpretation, and prove the set of all semantic values are accessible with respect to this ordering, and so all values are accessible so all types are well-founded.

Abel and Altenkirch also detail how to deal with mutual recursion and multiple arguments using call graphs and lexicographical ordering.

Definition 7. In a **call matrix** (a_{ij}) , the entry a_{ij} represents the relation $<$, $\leq$, or $?$ between callee argument i and caller argument j .

Addition on symbols is defined as combining information about a relation in parallel, while multiplication on symbols is defined as combining information about a relation serially.

$+$	$<$	$\leq$	$?$	$\cdot$	$<$	$\leq$	$?$
$<$	$<$	$<$	$<$	$<$	$<$	$<$	$?$
$\leq$	$<$	$\leq$	$\leq$	$\leq$	$<$	$\leq$	$?$
$?$	$<$	$\leq$	$?$	$?$	$?$	$?$	$?$

Definition 8. A **call graph** is a graph where vertices are functions and edges are calls between them, labeled by this call matrix. Multiple edges are possible.

Note that there is a natural composition operation on calls corresponding to matrix multiplication of the corresponding call matrices. Before termination checking, the call graph is fully saturated with all unique call matrices resulting from compositions of calls. Then each recursive function is checked with the structural ordering on its self-edges.

To deal with multiple arguments, a lexicographical ordering is used. A call must be found where at least one argument is strictly decreased to the parent call’s corresponding argument and the others are at most their corresponding argument. That argument is removed from consideration (including from the “others” check) and then the check is repeated until there are no more arguments left. This ensures that in all cases of recursive call an argument is decreased strictly. Abel and Altenkirch additionally prove this lexicographical ordering works with the corresponding interpretation of structural ordering in the proof that all types are well-ordered.

The authors admit a lack of treatment of dependent types in the paper, and set it as future work. They say this treatment requires more attention to checking the totality of pattern matching. In addition, the relation rules only being on elimination forms above means that termination checking where the case branches use the recursion result in introduction forms or constructors will not succeed. This is an expressivity limitation.

5 Bove and Capretta’s accessibility predicate

Bove and Capretta [5] observe that some general recursive algorithms cannot be expressed using structural recursion. The typical way to handle such algorithms is typically to use Nordström’s recursor, but proving the initial p is contained in $\text{Acc}(A, \prec)$ can be unwieldy with unnecessary and distracting additional code.

To solve this problem, the authors create a standard method of translating some general recursive functions to structural recursive functions on a proof term of termination (or as they frame it, proof of accessibility). More formally, the arguments $x_1 : A_1, x_2 : A_2, \dots, x_n : A_n$ to some recursive function f are extended with a term of type $\text{fAcc}(x_1, \dots, x_n)$ where $\text{fAcc} : (x_1 : A_1) \rightarrow \dots \rightarrow (x_n : A_n) \rightarrow \text{Set}$ with inductively defined constructors. fAcc is the proposition of termination on arguments $x_1, \dots, x_n$. Now, a typically non-structural recursive function can be turned structural recursive on the term of type $\text{fAcc}(x_1, \dots, x_n)$. The syntactic obligation of structural recursion is then satisfied in exchange for the new obligation of providing proof terms of termination of the initial call and for child recursive calls. This also allows for the translation of functions that only terminate on some inputs.

There are some restrictions on the recursive definitions. The functions that can be translated need to be fully applied with all arguments in recursive calls. Furthermore, if they cannot be expressed directly as structural

recursive functions without the addition of an `fAcc`, then they cannot be called in other functions because the extra argument causes some argument types to change.

Bove and Capretta define a general recursive calculus called $\mathcal{FP}$ with these restrictions and proving via Kleene’s normal form theorem that it is still able to express all partial recursive functions. They then formally describe how to translate programs in $\mathcal{FP}$ to programs in dependent type theory. For each case in the exhaustive case analysis on the arguments of the recursive function in $\mathcal{FP}$, there is one constructor of the `fAcc`. The authors define how to collect all the necessary context variables from the case analysis and other context used in the case result which is passed to this constructor. The constructor and its components are then included in that case in the case analysis in the translated version of the function.

The authors evaluate their solution by proving that a translated recursive function f is defined on its inputs if and only if `fAcc` is provable on those inputs, so long as the case result doesn’t have any lambda abstractions. Furthermore, they show that the translated f on its inputs and its proof of `fAcc` gives the same result as the f in $\mathcal{FP}$ applied to its inputs.

The limitations of the paper include the aforementioned inability to call the non-structural recursive functions from other functions and the inability to show that if f is defined on its inputs then `fAcc` is provable on its inputs when some case result has lambda abstractions. The authors address this in a later paper in 2004.

6 Barthe, et. al’s sized type-based termination

Barthe, et. al [3] observe several problems with structural recursion. Some terminating recursive algorithms are rejected, like Euclidean division, Ackermann’s function, or functions that swap arguments. Implementing and extending the “guard” condition for checking all recursive calls are structurally smaller is also difficult. Finally, the guard condition is often only expressed on values, not expressions, leading to inefficient termination checking from having to reduce terms. The authors also produce a counterexample of a non-terminating program that passes the termination check when expressions are normalized to values.

To solve this, the authors instead integrate termination checking into type checking by annotating types with sizes. They define a calculus $\hat{\lambda}$ with fixpoints and explicit constructors and size annotations.

Definition 9. A **stage** s is given by a stage variable ι , the successor of

another stage $\widehat{s}$, or the converged/empty stage ∞ .

Definition 10. A **datatype** d produces a type when applied to a stage and some type arguments, and is equipped with constructors $C(d)$.

Types are given by type variables, arrow types, and $d^s \vec{\tau}$.

$\widehat{\lambda}$ also has constructor schemes and instantiation of constructors $\text{Inst}_c^s \vec{\tau}$.

Stages can be compared by a binary relation $\preceq$ where $s \preceq \widehat{s}$ and $s \preceq \infty$.

Then there is a subtyping relation. The most interesting rule is

$$\frac{s \preceq r \quad \tau_i \sqsubseteq \tau'_i (\forall i)}{d^s \vec{\tau} \sqsubseteq d^r \vec{\tau}'} \text{Data}$$

This rule gives a subsumption principle; any term of a certain type given by a datatype with stage s is also a term of type given by a datatype with stage greater than s .

Constructor instantiation returns a value of type with increased stage:

$$\frac{}{\Gamma \vdash c : \text{Inst}_c^s \vec{\tau} \rightarrow d^{\widehat{s}} \vec{\tau}} \text{Constructor}$$

Case analysis provides the constructor components to each branch at a stage that is predecessor to the scrutinee's stage:

$$\frac{\Gamma \vdash e' : d^{\widehat{s}} \vec{\tau} \quad \Gamma \vdash e_i : \text{Inst}_{c_i}^s \vec{\tau} \rightarrow \theta (1 \leq i \leq n)}{\Gamma \vdash \text{case } e' \text{ of } \{c_1 \Rightarrow e_1 \mid \dots \mid c_n \Rightarrow e_n\} : \theta} \text{Case}$$

Finally, in a fixpoint, the recursive function is made available to call in the body, but with the obligation to provide arguments to the recursive call with lower stage than those of the current call. That is, if the current call's arguments have stage $\widehat{\iota}$ (as in e below), then recursive call is available in the context but must be provided arguments at stage ι (as in f below), and the definition of e is obligated to some rule like case analysis to generate such arguments at a lower stage.

$$\frac{\Gamma, f : d^{\iota} \vec{\tau} \rightarrow \theta \vdash e : d^{\widehat{\iota}} \vec{\tau} \rightarrow [\widehat{\iota}/\iota] \theta}{\Gamma \vdash (\text{letrec } f = e) : d^s \vec{\tau} \rightarrow [s/\iota] \theta} \text{Rec}$$

Barthe, et. al evaluate their solution by showing it is strictly stronger than structural recursion. They express Euclidean division and the Ackermann function in $\widehat{\lambda}$. Then, they prove some other metatheoretical properties. They give some standard proof of preservation. They also show strong normalization of all terms using semantic interpretation with saturated sets.

The authors then define a structural recursive calculus $\lambda_{\mathcal{G}}$ and embed it into $\widehat{\lambda}$, showing all well-typed terms in the former calculus are also well-typed in the latter.

The authors note that future work is required to scale up to polymorphism and dependent types; it has to be determined how annotations will be placed on these types. Second, $\widehat{\lambda}$ does not support type annotations at all. In that vein, checking and inference algorithms for not only types but also stages need to be developed. Finally, they note that additional work also has to be done to handle mutual recursion and mutually inductive types.

7 Blanqui’s application of sized types to dependent typing

Blanqui [4] notes that some higher-order rewrites in the Calculus of Algebraic Constructions need to be proven terminating. This is typically done manually with some techniques using transfinite iteration and induction. He also notes that existing sized types are only applied to simpler calculi with fixpoints and case analysis.

To solve this, Blanqui adds size annotations to the Calculus of Algebraic Constructions to allow for the automatic proof of rewrite termination. Blanqui slightly generalizes size annotations as in Barthe, et. al to allow more kinds of size terms in the size algebra. Size annotations are then only added to constant predicate types. As in Barthe, et. al, there is subtyping with lesser annotated types being subtypes of types with greater annotations.

A rewrite rule’s computability closure is the set of results r that are computable whenever inputs l are computable. Blanqui provides termination criterion based on the computability closure, well-typedness, linearity, accessibility, positivity, and safeness of the rule.

Blanqui evaluates his solution by proving termination of well-typed terms. He provides a semantic set interpretation of terms in the calculus. He then proves properties about monotonicity, that accessible subterms of a computable term are computable, that terms in the computability closure are computable, and ends with a proof that all well-typed terms are computable.

The limitations are mostly technical; the confluence of $\beta\cup\mathcal{R}$, that rewrite rules preserve typing, matching on defined symbols, and the decidability of type-checking. There is also the same limitation as in Barthe, et. al’s work where all types need to be provided with size annotations, where inference to reduce the burden of doing so on users would be helpful.

References

- [1] A. Abel and T. Altenkirch, “A predicative analysis of structural recursion,” *Journal of Functional Programming*, vol. 12, no. 12, 2000.
- [2] A. Abel, A. Vezzosi, and T. Winterhalter, “Normalization by evaluation for sized dependent types,” *Proc. ACM Program. Lang.*, vol. 1, no. ICFP, Aug. 2017. [Online]. Available: <https://doi.org/10.1145/3110277>
- [3] G. Barthe, M. J. Frade, E. Giménez, L. Pinto, and T. Uustalu, “Type-based termination of recursive definitions,” *Mathematical Structures in Computer Science*, vol. 14, no. 1, p. 97–141, 2004.
- [4] F. Blanqui, “A type-based termination criterion for dependently-typed higher-order rewrite systems,” in *Rewriting Techniques and Applications*, V. van Oostrom, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 2004, pp. 24–39.
- [5] A. Bove and V. Capretta, “Modelling general recursion in type theory,” *Mathematical Structures in Computer Science*, vol. 15, no. 4, p. 671–708, 2005.
- [6] B. Nordström, “Terminating general recursion,” *BIT Numerical Mathematics*, vol. 28, no. 3, pp. 605–619, Sep 1988. [Online]. Available: <https://doi.org/10.1007/BF01941137>
- [7] C. Paulin-Mohring, “Inductive definitions in the system coq - rules and properties,” in *International Conference on Typed Lambda Calculus and Applications*, 1993. [Online]. Available: <https://api.semanticscholar.org/CorpusID:27548688>

8 Updated Implementation Proposal

I plan to implement parts of Abel's fetus termination checker on some basic calculus, assuming I have already parsed it into an AST and extracted function calls. The implementation will likely be in OCaml.

There are three components described in the paper:

- The termination ordering search examines arguments of recursive calls and checks lexicographical ordering
- The call graph and matrices to handle mutual recursion
- Function call extraction from the AST

I will definitely implement the first component, since it is directly necessary for termination checking. I will attempt to get through implementing the second component, and will definitely not implement function call extraction.